

Experiment No.	7
Experiment Title	Dimensionality Reduction using PCA and Comparative Evaluation of Classification Models
Student Name	Chitraju Vishnu Vineeth
Register Number	3122247001012
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	31 August 2026

1 Objective

The aim of this experiment is to see how PCA affects different classifiers. Ten models, from simple linear ones to boosting and stacking ensembles, are trained on the same churn data with and without PCA. Each model is tuned with grid search under 5-fold cross-validation before the two settings are compared.

2 Problem Statement

Given 24 numerical attributes describing a customer (financial, usage and satisfaction scores), the task is to predict churn (1) or retention (0). Besides building a classifier, the experiment checks whether reducing these 24 features to a smaller set of principal components helps or hurts each model.

3 Dataset Description

Dataset Name	Customer Churn Dataset
Dataset Source	Provided course dataset (<code>customer_churn_dataset.csv</code>)
Number of Samples	150
Number of Features	24 numerical features (target column Churn excluded)
Number of Classes	2 (0 = Retained, 1 = Churned)
Missing Values	None (0 missing entries across all 25 columns)
Train-Test Split	80% train / 20% test, stratified on Churn (120 train, 30 test samples)

4 Exploratory Data Analysis

Dataset Overview and Statistical Summary

Every column in the file is already numeric and continuous, so no categorical encoding is needed. The features live on wildly different scales, however: `monthly_income` runs into the thousands, `loyalty_points` into the hundreds, while `satisfaction_rating` and `referral_count` sit in single digits. Since PCA works on the covariance structure and is scale-dependent, standardization is mandatory here rather than optional, and the same applies to the distance and margin based models used later.

Missing Value Analysis

A column-wise `isnull().sum()` returns zero for all 25 columns, and the overall total is also 0. No imputation step was therefore required.

Class Distribution

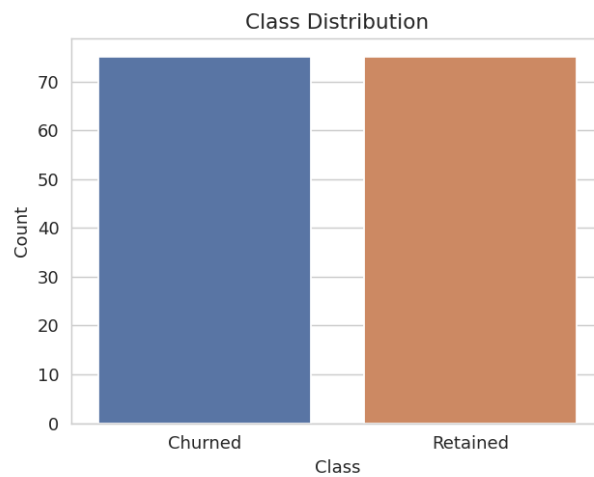


Figure 1: Distribution of retained and churned customers.

Inference: The classes are perfectly balanced, 75 retained and 75 churned. F1-score was still used for tuning since it focuses on the churned class specifically.

Correlation Matrix

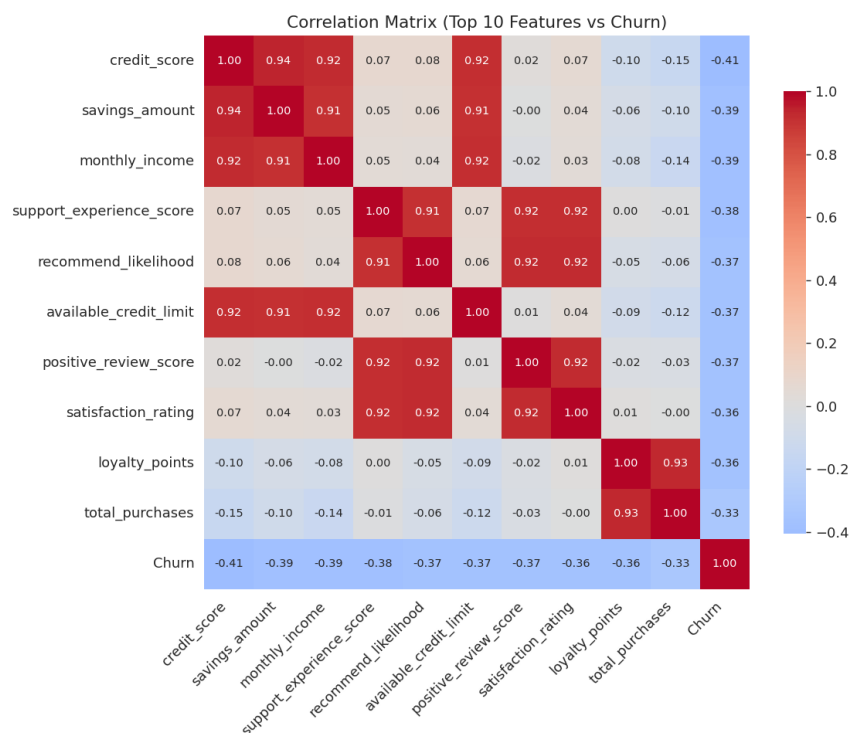


Figure 2: Correlation heatmap of the 10 features most correlated with Churn, with values shown.

Inference: Financial features (`credit_score`, `savings_amount`, `monthly_income`) correlate strongly with each other (0.91-0.94) and each carries about -0.4 correlation with Churn. Satisfaction-related scores show a similar pattern, so a lot of this information is repeated across columns.

Feature Distribution

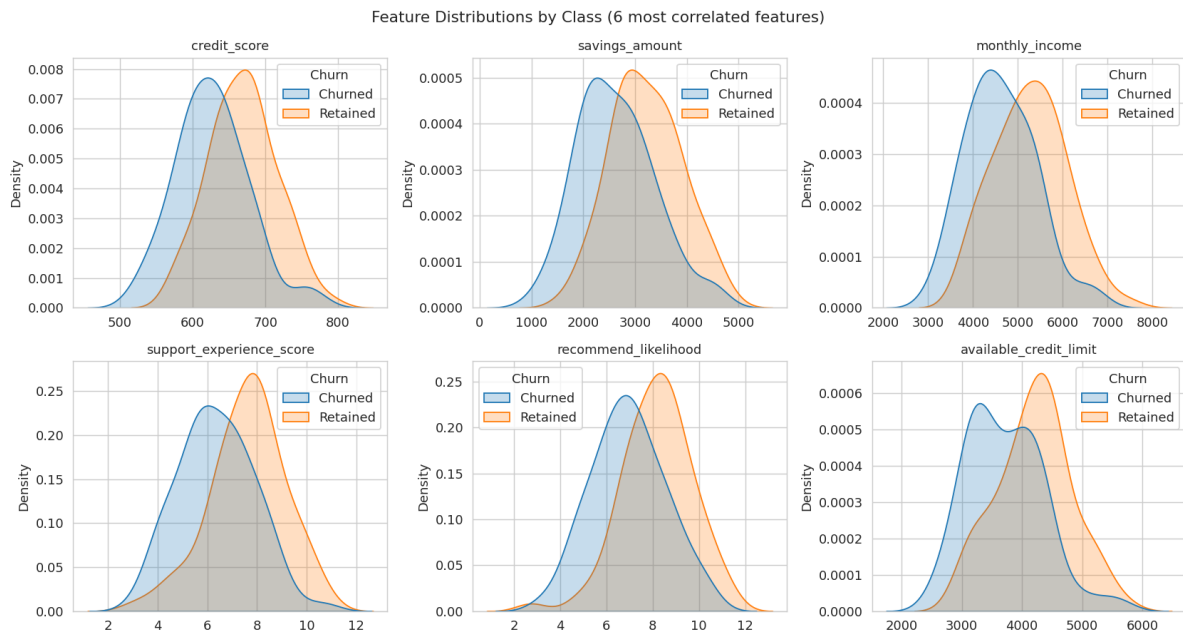


Figure 3: Class-conditional distributions of the six features most correlated with churn.

Inference: The class curves are offset for most features but still overlap in the middle, so no single feature separates the classes on its own.

5 Data Preprocessing

- **Missing value handling:** Not needed; the dataset contains no missing entries.
- **Encoding:** Not needed, as all 24 predictors are already numeric and the target is already 0/1.
- **Feature scaling:** `StandardScaler` was applied to bring all features to zero mean and unit variance. Importantly, the scaler was placed *inside* the scikit-learn `Pipeline` rather than fitted once on the whole training set, so that within every cross-validation fold the scaler sees only that fold's training portion. This prevents leakage of validation-fold statistics into the fitted model.
- **Dimensionality reduction:** For the With-PCA setting, a `PCA(n_components=0.95)` step was inserted into the pipeline between scaling and the model, again so that PCA is refitted inside every fold.
- **Train-test split:** 80/20 stratified split with `random_state=42`, giving 120 training and 30 test samples.

6 Machine Learning Algorithm

Principal Component Analysis

PCA is an unsupervised linear transformation that finds the directions of maximum variance in the data. After centering, it computes the eigenvectors of the covariance matrix $\Sigma = \frac{1}{n-1}X^T X$, and projects the data onto the top k eigenvectors:

$$Z = XW_k, \quad \Sigma w_i = \lambda_i w_i$$

where the eigenvalue λ_i gives the variance captured by component i . Components are ordered by decreasing λ , and k is chosen so the cumulative explained variance ratio crosses a target threshold, 95% in this experiment.

Advantages: Removes redundancy from correlated features, reduces the number of dimensions a model must fit, and can act as a form of noise filtering.

Limitations: The components are linear combinations of all original features, so interpretability is lost. It is also unsupervised, meaning it maximizes variance without any knowledge of the class label, so a low-variance direction that happens to be highly discriminative can be discarded.

Classification Models Compared

Ten classifiers were evaluated, spanning four families:

- **Linear / margin-based:** Logistic Regression (sigmoid of a linear score, minimizing log-loss) and SVM (maximum-margin hyperplane, with linear and RBF kernels searched).
- **Probabilistic / instance-based:** Gaussian Naive Bayes (assumes conditional independence of features) and KNN (majority vote among the k nearest training points).
- **Tree-based and bagging:** Decision Tree and Random Forest.
- **Boosting and stacking:** AdaBoost, Gradient Boosting, XGBoost, and a Stacking ensemble combining SVM, KNN and a depth-5 Decision Tree as base learners with Logistic Regression as the meta-learner.

This spread was chosen deliberately: PCA affects each family differently, since some models depend on distances between points, some on axis-aligned splits, and some only on a linear score.

7 Experimental Setup

Python Version	3.12.3
Scikit-Learn Version	1.8.0
XGBoost Version	3.4.1
IDE	Jupyter Notebook
Operating System	Windows 11
Hardware	Intel Core i5, 8 GB RAM

8 Hyperparameter Selection

Every model was tuned with `GridSearchCV` using `StratifiedKFold` with `n_splits=5`, `shuffle=True`, `random_state=42`, and `scoring="f1"`. The identical grid was searched twice per model, once for the No-PCA pipeline and once for the With-PCA pipeline, so any difference in results comes from the PCA step and not from an unequal search.

Model	Parameter	Values searched
SVM	C / kernel / gamma	{0.1, 1, 10} / {linear, rbf} / {scale, auto}
Naive Bayes	var_smoothing	{1e-9, 1e-8, 1e-7}
KNN	n_neighbors / weights / metric	{3, 5, 7} / {uniform, distance} / {euclidean, manhattan}
Logistic Regression	C / solver	{0.1, 1, 10} / {liblinear}
Decision Tree	max_depth / min_samples_split	{None, 5, 10} / {2, 5}
Random Forest	n_estimators / max_depth	{100, 200} / {None, 10}
AdaBoost	n_estimators / learning_rate	{50, 100} / {0.5, 1.0}
Gradient Boosting	n_estimators / lr / max_depth	{50, 100} / {0.05, 0.1} / {2, 3}
XGBoost	n_estimators / lr / max_depth	{50, 100} / {0.05, 0.1} / {2, 3}
Stacking	final_estimator__C	{0.1, 1, 10}

Table 2: Hyperparameter search space (identical for both settings).

PCA Component Selection

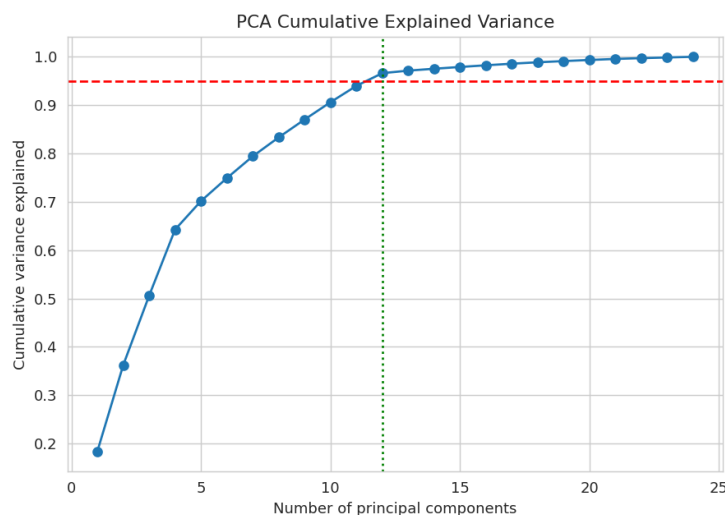


Figure 4: Cumulative explained variance against number of principal components.

Inference: The curve crosses 95% at 12 components (96.61% variance), so half the original features can be dropped with little information loss.

Setting	Chosen nents / Target	Compo- nents / Variance	Explained Variance (%)	Justification
With-PCA	12 / 95%		96.61	Reduce 24 original features (several highly correlated / redundant) to 12 components while retaining approximately 95% variance

Table 3: PCA Variance Explained.

9 Hyperparameter Tuning Templates

Each model below was tuned with `GridSearchCV` (5-fold, `scoring="f1"`) using the search space and pipeline described in Section 8. The tables report the best configuration found and the resulting best cross-validation F1 for the No-PCA and With-PCA pipelines.

Support Vector Machine (SVM)

Kernel	C Tried	Values	Gamma Values Tried	Performance (No-PCA)	Performance (With-PCA)
{linear, rbf}	{0.1, 1, 10}		{scale, auto}	0.8682 (rbf, C=0.1, scale)	0.8625 (rbf, C=0.1, scale)

Table 4: SVM — Hyperparameter Tuning Results. Performance is the best cross-validation F1, with the winning configuration in parentheses.

Naïve Bayes

Smoothing Parameter	Performance (No-PCA)	Performance (With-PCA)
{1e-9, 1e-8, 1e-7}	0.8346 (var_smoothing=1e-9)	0.8111 (var_smoothing=1e-9)

Table 5: Naïve Bayes — Smoothing Choices.

KNN

k Values	Weights	Distance Metrics	Performance (No-PCA)	Performance (With-PCA)
{3, 5, 7}	{uniform, distance}	{euclidean, manhattan}	0.8116 ($k=5$, uniform, manhattan)	0.8099 ($k=7$, uniform, euclidean)

Table 6: KNN — Hyperparameter Tuning.

Logistic Regression

C Tried	Values	Solver	Performance (No- PCA)	Performance (With-PCA)
{0.1, 1, 10}		{liblinear}	0.8470 (C=0.1)	0.8539 (C=0.1)

Table 7: Logistic Regression — Hyperparameter Tuning.

Decision Tree

Max Depth	Min Split	Samples	Performance (No-PCA)	Performance (With-PCA)
{None, 5, 10}	{2, 5}		0.7769 (None, 2)	0.7357 (5, 5)

Table 8: Decision Tree — Hyperparameter Tuning.

Random Forest

n_estimators	Max Depth	Performance (No-PCA)	Performance (With-PCA)
{100, 200}	{None, 10}	0.8490 (200, None)	0.8199 (100, None)

Table 9: Random Forest — Hyperparameter Tuning (min_samples_split {2, 5} also searched; best was 2 for No-PCA and 5 for With-PCA).

AdaBoost

n_estimators	Learning Rate	Performance (No-PCA)	Performance (With-PCA)
{50, 100}	{0.5, 1.0}	0.8564 (50, 1.0)	0.8300 (50, 1.0)

Table 10: AdaBoost — Hyperparameter Tuning.

Gradient Boosting

n_estimators	Learning Rate	Max Depth	Performance (No-PCA)	Performance (With-PCA)
{50, 100}	{0.05, 0.1}	{2, 3}	0.8505 (100, 0.1, 2)	0.8237 (50, 0.1, 2)

Table 11: Gradient Boosting — Hyperparameter Tuning.

XGBoost

n_estimators	Learning Rate	Max Depth	Performance (No-PCA)	Performance (With-PCA)
{50, 100}	{0.05, 0.1}	{2, 3}	0.8598 (100, 0.1, 3)	0.8208 (100, 0.1, 3)

Table 12: XGBoost — Hyperparameter Tuning.

Stacking

Final Estimator C	Performance (No-PCA)	Performance (With-PCA)
{0.1, 1, 10}	0.8624 (C=0.1)	0.8510 (C=10)

Table 13: Stacking — Hyperparameter Tuning. Base learners (SVM, KNN, Decision Tree) are fixed; only the meta-learner’s C is tuned.

10 Results

Model	Acc (No PCA)	Acc (PCA)	F1 (No PCA)	F1 (PCA)	$\Delta F1$
SVM	0.9000	0.9000	0.9091	0.9091	+0.0000
Naive Bayes	0.8667	0.9000	0.8750	0.9032	+0.0282
KNN	0.8667	0.8667	0.8824	0.8750	−0.0074
Logistic Regression	0.9000	0.9000	0.9032	0.9032	+0.0000
Decision Tree	0.7333	0.8333	0.7647	0.8387	+0.0740
Random Forest	0.8333	0.8667	0.8571	0.8824	+0.0252
AdaBoost	0.8000	0.8667	0.8125	0.8750	+0.0625
Gradient Boosting	0.8333	0.8333	0.8571	0.8485	−0.0087
XGBoost	0.7667	0.8667	0.8000	0.8824	+0.0824
Stacking	0.8667	0.9000	0.8824	0.9032	+0.0209

Table 14: Test-set performance of all ten models, with and without PCA.

Model	Best CV F1 (No PCA)	Best CV F1 (With PCA)
SVM	0.8682	0.8625
Naive Bayes	0.8346	0.8111
KNN	0.8116	0.8099
Logistic Regression	0.8470	0.8539
Decision Tree	0.7769	0.7357
Random Forest	0.8490	0.8199
AdaBoost	0.8564	0.8300
Gradient Boosting	0.8505	0.8237
XGBoost	0.8598	0.8208
Stacking	0.8624	0.8510

Table 15: Best 5-fold cross-validation F1 from grid search.

Best Model Detail

The best overall performer was the tuned SVM (`C=0.1`, `kernel=rbf`, `gamma=scale`), identical in both settings:

Metric	Value (SVM)
Accuracy	0.9000
Precision (Churned)	0.8300
Recall (Churned)	1.0000
F1-score (Churned)	0.9091
ROC-AUC	0.9333

Table 16: Detailed metrics for the best model on the 30-sample test set.

5-Fold Cross-Validation Results (All Models)

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg (No-PCA)	Avg (With-PCA)
SVM	1.000	0.769	0.769	0.957	0.846	0.8682	0.8625
Naive Bayes	1.000	0.750	0.667	0.957	0.800	0.8346	0.8111
KNN	0.923	0.741	0.800	0.880	0.714	0.8116	0.8099
Logistic Regression	0.957	0.769	0.720	0.909	0.880	0.8470	0.8539
Decision Tree	0.857	0.727	0.800	0.833	0.667	0.7769	0.7357
Random Forest	0.909	0.800	0.833	0.917	0.786	0.8490	0.8199
AdaBoost	1.000	0.696	0.870	0.917	0.800	0.8564	0.8300
Gradient Boosting	0.870	0.833	0.870	0.800	0.880	0.8505	0.8237
XGBoost	0.917	0.800	0.917	0.880	0.786	0.8598	0.8208
Stacking	0.923	0.800	0.783	0.960	0.846	0.8624	0.8510

Table 17: 5-Fold Cross-Validation Results (No-PCA vs With-PCA). Fold columns show the No-PCA run's per-fold F1; both average columns are the best cross-validation F1 returned by `GridSearchCV` for that setting.

Per-Fold Detail for Selected Models

Model	Setting	F1	F2	F3	F4	F5	Average
SVM	No-PCA	1.000	0.769	0.769	0.957	0.846	0.8682
SVM	With-PCA	1.000	0.769	0.741	0.957	0.846	0.8625
XGBoost	No-PCA	0.917	0.800	0.917	0.880	0.786	0.8598
XGBoost	With-PCA	0.917	0.769	0.769	0.957	0.692	0.8208
Stacking	No-PCA	0.923	0.800	0.783	0.960	0.846	0.8624
Stacking	With-PCA	0.957	0.800	0.720	0.909	0.870	0.8510

Table 18: Fold-wise F1 scores for three representative models ($K = 5$).

Stability Across Folds

Model	Std (No PCA)	Std (With PCA)	More Stable
SVM	0.0951	0.1015	No PCA
Naive Bayes	0.1255	0.0846	With PCA
KNN	0.0797	0.0758	With PCA
Logistic Regression	0.0884	0.1103	No PCA
Decision Tree	0.0704	0.0623	With PCA
Random Forest	0.0545	0.1193	No PCA
AdaBoost	0.1034	0.1062	No PCA
Gradient Boosting	0.0298	0.1216	No PCA
XGBoost	0.0565	0.0994	No PCA
Stacking	0.0734	0.0832	No PCA

Table 19: Standard deviation of fold-wise F1 scores.

11 Result Visualization

Accuracy Comparison

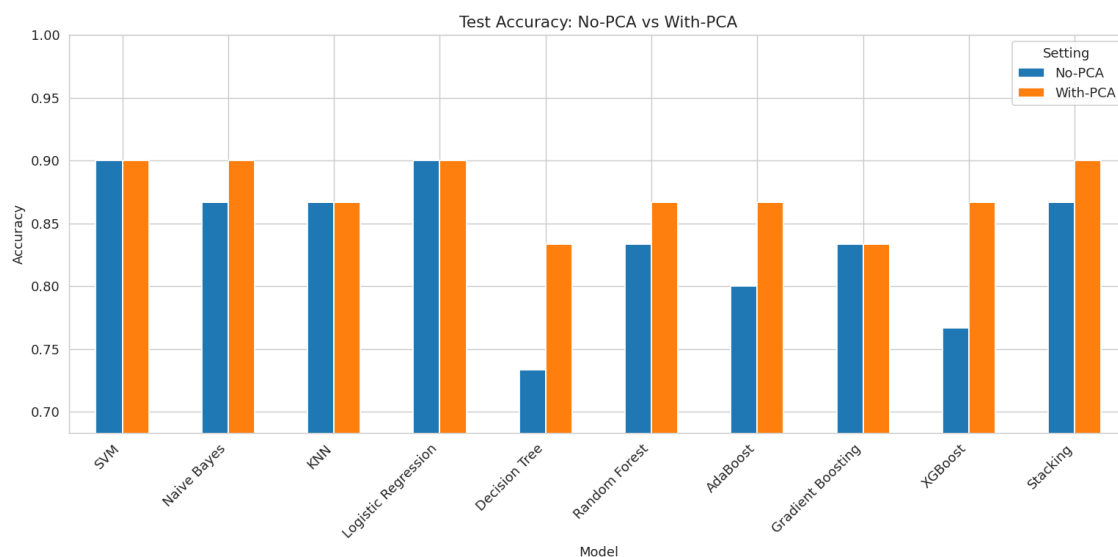


Figure 5: Test accuracy of each model with and without PCA.

Inference: PCA matches or beats No-PCA on seven of ten models. The biggest jumps are XGBoost (0.767 to 0.867) and Decision Tree (0.733 to 0.833); SVM, Logistic Regression and KNN barely move.

F1-Score Comparison

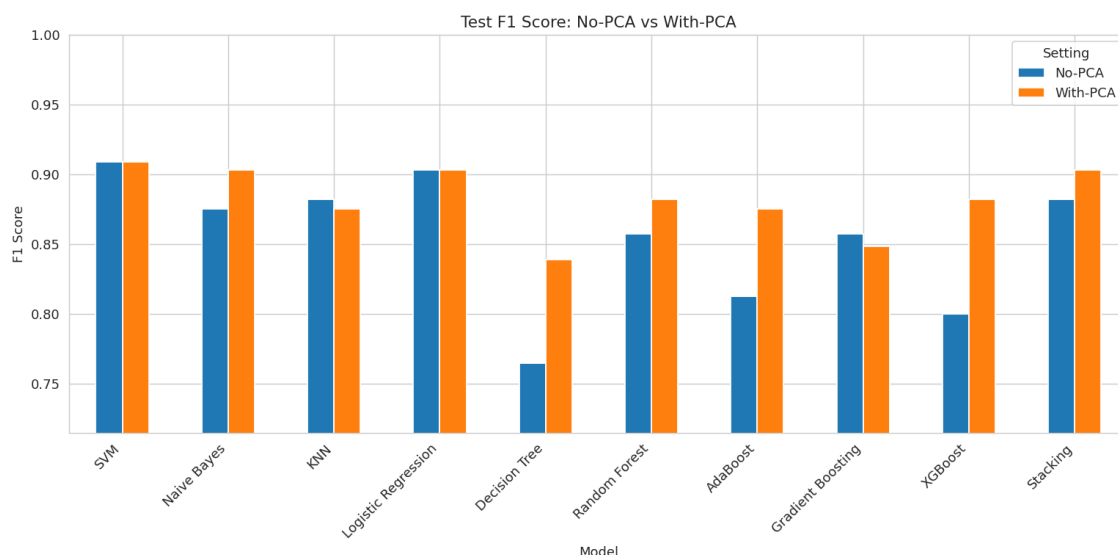


Figure 6: Test F1-score of each model with and without PCA.

Inference: XGBoost gains the most F1 (+0.082), followed by Decision Tree (+0.074). KNN and Gradient Boosting drop slightly. After PCA, four models sit close to 0.90 F1.

Fold Stability

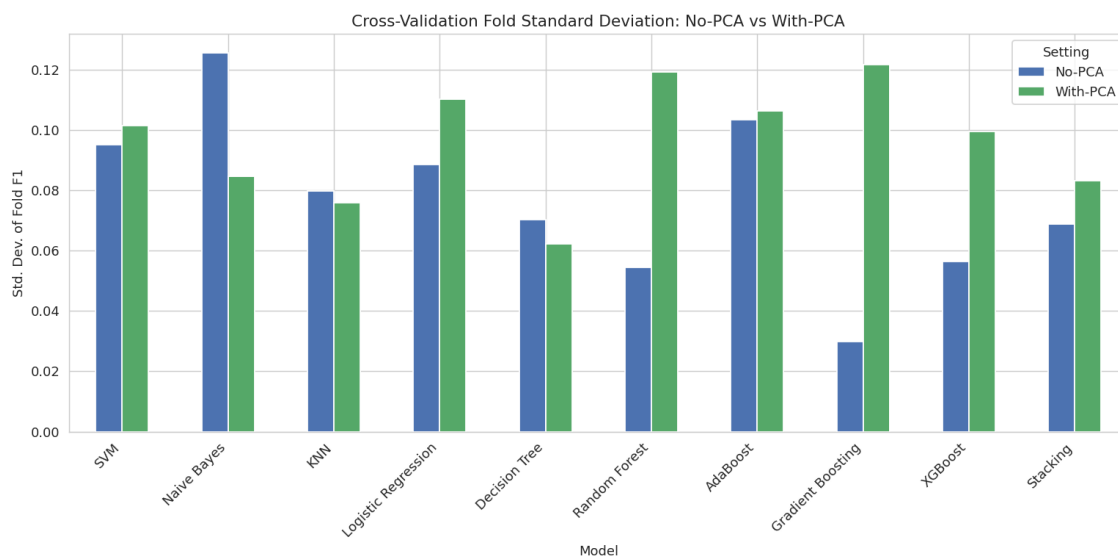


Figure 7: Standard deviation of fold-wise F1 scores for each model under both settings.

Inference: Only 3 of 10 models (Naive Bayes, KNN, Decision Tree) get more stable with PCA; the other 7 get more variable. So PCA did not consistently reduce fold-to-fold variance here.

Confusion Matrices

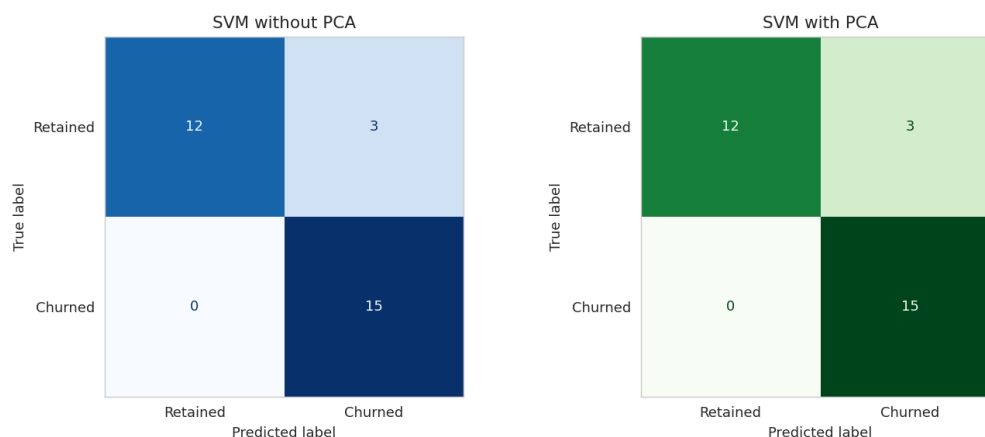


Figure 8: Confusion matrices for the tuned SVM without PCA (left) and with PCA (right).

Inference: Both settings give the same result: all 15 churned customers are caught, and 3 of 15 retained customers are wrongly flagged. PCA did not change the SVM's predictions at all.

ROC Curve

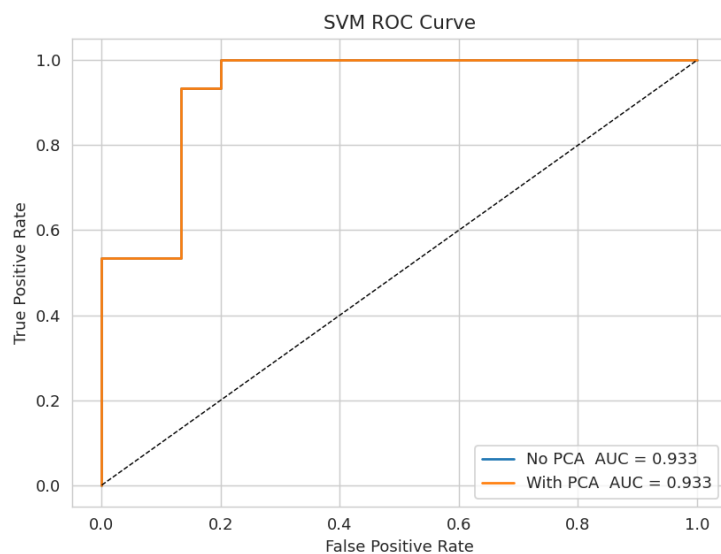


Figure 9: ROC curves for the SVM under both settings.

Inference: Both curves give an AUC of 0.933, so PCA does not change how well the SVM ranks customers by churn risk.

Precision-Recall Curve

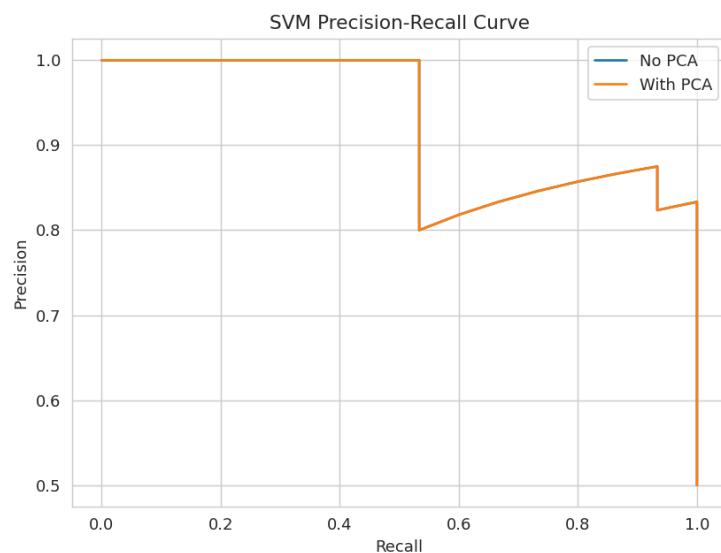


Figure 10: Precision-Recall curves for the SVM under both settings.

Inference: Precision stays high until recall nears 1, matching the confusion matrix where full recall came with 3 false positives.

12 Discussion

1. Which models improved most with PCA, which did not, and why?

PCA helped Naive Bayes, Decision Tree, Random Forest, AdaBoost, XGBoost and Stacking. XGBoost improved the most, F1 went from 0.8000 to 0.8824. Decision Tree also improved a lot, from 0.7647 to 0.8387. KNN and Gradient Boosting dropped slightly, and SVM and Logistic Regression barely changed. PCA does not help every model equally because reducing dimensions can also remove information some algorithms were actually using.

2. Did PCA reduce variance across folds (more stable results)?

Not really. The fold results did not become more consistent for every model after PCA. Some models were more stable, others less, so PCA cannot be said to make all models more stable here. Its main benefit in this experiment was better test scores for a few models, not steadier cross-validation.

3. Was PCA useful for reducing overfitting on this high-dimensional data?

Yes, for some models. Decision Tree, XGBoost, AdaBoost, Random Forest and Stacking all had better test performance with PCA, which suggests the smaller feature set let them generalize better instead of fitting noise in the extra columns. PCA did not reduce overfitting the same way for every model, though.

4. How did linear models (Logistic Regression, SVM) compare to ensembles under PCA?

The linear models barely reacted to PCA: SVM stayed at $F1 = 0.9091$ and Logistic Regression at $F1 = 0.9032$ in both settings, since they were already fitting a good linear boundary with all 24 features. Ensembles changed much more, e.g. XGBoost went from 0.8000 to 0.8824 and AdaBoost from 0.8125 to 0.8750. So PCA had a bigger effect on ensemble models than on the two linear ones.

5. Did Stacking show robustness to dimensionality reduction?

Yes. Stacking's test F1 rose from 0.8824 without PCA to 0.9032 with PCA, and accuracy from 0.8667 to 0.9000. Its cross-validation F1 changed only slightly (0.8624 to 0.8510). Combining SVM, KNN and a Decision Tree inside one ensemble seems to have made Stacking handle the reduced feature space well.

Limitations

The test set has only 30 samples, so one misclassified customer changes accuracy by about 3%, and small differences between models should not be read as conclusive. A larger dataset would make these comparisons more reliable.

13 Conclusion

This experiment applied PCA to a 24-feature customer churn dataset and compared ten tuned classifiers with and without it. Twelve components kept 96.61% of the variance, halving the feature count. The tuned RBF-SVM was the best model overall (0.90 accuracy, 0.9091 F1, 0.933 ROC-AUC), unchanged by PCA. Tree-based and boosting models gained the most from PCA, especially XGBoost (+0.0824 F1) and Decision Tree (+0.0740 F1), while the two linear models were unaffected. PCA did not consistently make results more stable across folds. Overall, PCA helped selectively rather than universally, so it is worth validating per model rather than applying by default.

14 References

- [1] Scikit-learn developers, “Decomposing signals in components (matrix factorization problems),” *scikit-learn documentation*. [Online]. Available: <https://scikit-learn.org/stable/modules/decomposition.html>
- [2] Scikit-learn developers, “Ensemble methods,” *scikit-learn documentation*. [Online]. Available: <https://scikit-learn.org/stable/modules/ensemble.html>
- [3] Scikit-learn developers, “Tuning the hyper-parameters of an estimator,” *scikit-learn documentation*. [Online]. Available: https://scikit-learn.org/stable/modules/grid_search.html